Database Migrations and Safe Rollbacks

Day 05 | AWS / AZURE / GCP / APP + POSTGRESQL

Use this guide to investigate a specific failure, apply a cause-specific change and verify recovery. Start with evidence from the affected workload. The examples require adaptation to your environment.

How the failure happens

Code rollback and data recovery are separate operations. Additive schema changes preserve a compatibility window in which both old and new application versions can run. Destructive cleanup belongs after the window closes and the migration is validated.

The Scenario A release drops a database column. The previous image still queries it.

Why Rollback Fails Restoring an image does not restore the old schema or its data.

The Pattern Expand, migrate, then contract after the compatibility window closes.

Before you start

A staging copy of the schema; current and previous application builds; a reviewed migration tool; tested backup and restore; realistic table sizes.

Diagnostic checks

```
-- Inspect the expected schema
SELECT column_name
FROM information_schema.columns
WHERE table_name = 'customers';
```

Check Both Sides: Compare old and new application queries with the deployed schema.

Do Not Guess: A destructive down migration can lose data or block writes.

Apply the cause specific fix

Fix The Confirmed Cause

```
-- Expand first; illustrative PostgreSQL SQL
ALTER TABLE customers
  ADD COLUMN display_name text;

-- Deploy compatible code and backfill safely.
-- Keep the old column during rollback window.
-- Drop only in a later reviewed migration.
```

Safe Transition: Support both schema shapes while migrating reads and writes. Backfill in controlled batches.

Contract Later: Remove old fields only after older application versions are no longer required.

Validate the result

```
-- In a staging copy of the real schema:
-- Run old-version and new-version app tests.
-- Verify migrated values and rollback behaviour.
```

Expected: The previous compatible version still functions during the rollback window.

Measure: Review locks, migration duration, error rates and data correctness.

Practice exercise

Add the new column in a lab. Run both application versions, backfill representative data and confirm the old version remains viable. Schedule destructive cleanup separately.

Implementation notes

SQL is illustrative, assumes a customers table, and is not an executable migration plan. Large tables need lock/statement-timeout planning and tested backfills. Backup restoration and forward fixes have different RPO/RTO impacts. Do not restore production databases blindly after valid new transactions.

Interview question

Why can reverting a Deployment fail to recover the application? The previous code can still depend on schema or data removed by a migration.

Record what changed

- The symptom and affected user operation.
- Evidence supporting the root cause and alternatives ruled out.
- The changed configuration and the version or identity involved.
- Positive and negative tests, plus any remaining risks.

Example status

Examples are educational templates or fragments. They were not deployed against a live cloud account or production cluster. A successful syntax check is not proof of runtime compatibility. Test adapted configurations in an isolated environment first.

Official references

Official documentation 1